Real-world (persist and read back real data) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `sqlite_persistence_practice.py`: this is the Build Task's run history pattern in miniature — save every request's input and output to SQLite, then read one back later by its ID. Practicing it first on a one-line fake chat function keeps the SQL itself as the only genuinely new thing to learn. **If not:** the Build Task's `runs` table, its `GET /runs/{id}` route, and its 404 case would all be first attempts, tangled up with a real multi-agent pipeline at the same time.

The solution below creates `runs.db` in `practice/` the first time it runs. Delete that file between tries if you want a clean slate.

Solution

Story: code reading `row[2]` has to know the table's exact column order by heart, and breaks silently the day a column is added or reordered. Converting each row to named fields (`dict(row)` or a Pydantic model) fixes that for good. **If not:** the Build Task's `GET /runs/{id}` would hand back a bare positional list, and no `response_model` could ever check or document it.

```
# practice/sqlite_persistence_practice.py — Real-world section
import sqlite3
from datetime import datetime, timezone
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
# how: every row now behaves like a dict — row["input"] works
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        created_at TEXT
    )
""")
```

```
class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    reply = "You said: " + request.message
    now = datetime.now(timezone.utc).isoformat()
    # why: ? placeholders — values are never pasted into the SQL text
    conn.execute(
        "INSERT INTO runs (input, output, created_at) VALUES (?, ?, ?)",
        (request.message, reply, now),
    )
    conn.commit()
    return {"reply": reply}

@app.get("/runs")
def get_runs():
    rows = conn.execute("SELECT * FROM runs").fetchall()
    result = []
    for row in rows:
        # how: dict(row) turns a sqlite3.Row into a real dict
        result.append(dict(row))
    return result
```

Expected output (`GET /runs`; shown wrapped onto 2 lines just to fit the page — really one line of output):

```
[{"id":1,"input":"hi","output":"You said: hi",
  "created_at":"2026-01-01T00:00:00.000000+00:00"}]
```